

Міністерство освіти і науки України
Національному університеті "Львівська
Політехніка

Кафедра систем штучного інтелекту



Лабораторна робота №1

з дисципліни

«Інтелектуальний аналіз та візуалізація даних»

Виконала:

студентка групи ІІІ-31 Потапова Світлана

Прийняла:

Торська Р. В.

Львів – 2026

Лабораторна робота № 1

Вхідний аналіз набору даних

Дисципліна: Інтелектуальний аналіз та візуалізація даних (СК26) **Тривалість:** 2 год аудиторно + 3 год самостійної роботи · **Максимальна оцінка:** 4 бали

Мета роботи

Пригадати роботу в Jupyter Notebook, бібліотеці `pandas` та побудову графіків — і зробити це не на навчальному прикладі, а на реальному файлі, який ніхто не готував спеціально для вас.

Ця робота виконується **до першої лекції**. Нічого нового знати не потрібно: усе, що знадобиться, ви вже проходили в попередніх курсах. Усі поняття, яких ви тут торкнетесь мимохідь, отримають назви й пояснення на лекціях 3 і 4.

Дані

Файл `20171129_FINAL_FINAL.xlsx`, аркуш `Sheet1` — спостереження космічної погоди за 28 серпня — 22 вересня 2017 року, зібрані з трьох різних джерел.

Файл видається як є, рівно в тому вигляді, у якому його колись вивантажили. Ніхто не приводив його до ладу, і саме в цьому сенс: справжні дані здебільшого надходять саме такими.

Що ви маєте зробити

Етап	Зміст	Де
1	Розібратися, що взагалі лежить у файлі	аудиторно
2	Виділити три таблиці й привести їх до робочого вигляду	аудиторно
3	Скласти паспорт даних: що є, чого бракує, що виглядає дивно	аудиторно
4	Об'єднати три таблиці в одну	аудиторно
5	Побудувати п'ять графіків, кожен — відповідь на питання	аудиторно + самостійно
6	Порівняти два графіки одних і тих самих даних	самостійно
7	Записати три спостереження про дані	самостійно
8	Декларація про використання інструментів ШІ	самостійно

Що здається

Цей файл `.ipynb` із заповненими розв'язками. **Ноутбук має виконуватися згори донизу на чистому ядрі (Kernel Restart & Run All) без помилок**

згори донизу на чистому ядрі (Kernel → Restart & Run All) без помилок — це перевіряється першою дією на захисті.

Файл `master_hourly.csv` — об'єднана таблиця з етапу 4.

Експорт ноутбука у HTML або PDF.

Заповнена декларація (етап 8). Без неї фінальна самоперевірка не проходить.

Критерії оцінювання

Бали	Що досягнуто
0	Дані не опрацьовано, графіки не побудовано або побудовано з грубими помилками
1	Таблиці прочитано й кілька графіків побудовано, але типи графіків не відповідають даним, дивних знач
2	Таблиці розібрано й об'єднано, графіки коректні та підписані, але висновків під ними немає або вони ф
4	Усе вище виконано, знайдено й пояснено дивні значення, під кожним графіком є змістовний висновок

Головне про оцінку: бали ставляться не за наявність графіка, а за речення під ним. Код, який нічого не пояснює, коштує двох балів з чотирьох.

Нагадування про Jupyter

Якщо давно не працювали — три речі, які знадобляться сьогодні:

- `Shift+Enter` виконує клітинку, `Esc A / Esc B` додає клітинку вище / нижче, `Esc M` перетворює клітинку на текстову (Markdown), `Esc Y` — назад на код.
- **Порядок виконання має значення.** Якщо ви правили клітинки в довільному порядку, результат може залежати від історії запусків. Перед здачею обов'язково **Kernel → Restart & Run All** — так ви побачите ноутбук очима викладача.
- Текстові клітинки — не прикраса. Висновки пишуться в них, а не в коментарях до коду.

Крок 0. Ваші дані та варіант

ІНДИВІДУАЛЬНЕ ЗАВДАННЯ • Потапова Світлана, ШІ-31, варіант 16

=====

ваш потік: P>5

ваша пара для графіка 3: радіопотік F10.7 та швидкість вітру ваш агрегат

для етапу 4: максимум ваше додаткове питання: У яку добу середнє

значення вашого потоку було н

Далі за текстом «ваш потік» означає стовпець названий у першому рядку

Далі за текстом «ваш потік» означає стовпець, названий у першому рядку завдання.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

pd.set_option('display.width',200)
pd.set_option('display.max_columns',40)
plt.rcParams.update({'figure.dpi':110,'axes.spines.top':False,
                    'axes.spines.right':False,'font.size':9})

ФАЙЛ = '20171129_FINAL_FINAL.xlsx'
```

Етап 1. Що взагалі лежить у файлі

Перше правило роботи з незнайомими даними: спершу подивитися, потім рахувати.

Завдання 1.1

Спробуйте прочитати аркуш Sheet1 звичайним способом — `pd.read_excel(ФАЙЛ)` — і виведіть перші рядки та типи стовпців.

Очікуваний результат: результат буде дивним. Подивіться на назви стовпців і на те, які там типи даних. Поясніть у наступній клітинці, чому так вийшло.

```
# ВАШ КОД
df = pd.read_excel(ФАЙЛ)

display(df.head())

print(df.dtypes)
```

	Unnamed: 0	ftp://ftp.swpc.noaa.gov/pub/lists/particle/	Unnamed: 2	Unnamed: 3	Unnamed: 4	Unnamed: 5	U
0	NaN	NaN	NaN	NaN	NaN	NaN	
1	NaN	:Data_list: 20170901_Gs_part_5m.txt	NaN	NaN	NaN	NaN	
2	NaN	:Created: 2017 Sep 02 0011 UTC	NaN	NaN	NaN	NaN	
		0011 UTC	2	NaN	NaN	NaN	NaN

Prepared by the U.S.

3 NaN Dept. of Commerce, NaN NaN NaN NaN
NOAA...

Please send comments

4 NaN and suggestions to NaN NaN NaN NaN
SWPC...

```
Unnamed: 0 float64
ftp://ftp.swpc.noaa.gov/pub/lists/particle/ object
Unnamed: 2 object
Unnamed: 3 object
Unnamed: 4 object
Unnamed: 5 object
Unnamed: 6 object
Unnamed: 7 object
Unnamed: 8 object
Unnamed: 9 object
Unnamed: 10 object
Unnamed: 11 object
Unnamed: 12 object
Unnamed: 13 object
Unnamed: 14 object
Unnamed: 15 float64
Unnamed: 16 object
Unnamed: 17 object
Unnamed: 18 object
Unnamed: 19 object
Unnamed: 20 float64
Unnamed: 21 float64
Unnamed: 22 float64
Unnamed: 23 float64
Unnamed: 24 float64
Unnamed: 25 float64
http://umtof.umd.edu/pm/crn/ object
Unnamed: 27 object
Unnamed: 28 object
Unnamed: 29 object
Unnamed: 30 object
Unnamed: 31 object
dtype: object
```

Відповідь 1.1: (чому звичайне читання дало саме такий результат)

Результат виявився дивним через те, що на початку файлу містяться службові текстові рядки, а не сама таблиця. Замість заголовків у першому рядку знаходяться посилання та пусті комірки, тому більшість стовпців мають назви unnamed, а а через наявність тексту в рядках самі стовпці набули типу object

Завдання 1

Прочитайте аркуш ще раз, але **без заголовка**: `pd.read_excel(ФАЙЛ, header=None)` .Виведіть його розмір і подивіться на перші 30 рядків.

11	NaN	# Label: P >50 = Particles at >50 Mev	NaN	NaN	NaN	NaN	NaN	NaN	NaN	N
12	NaN	# Label: P>100 = Particles at >100 Mev	NaN	NaN	NaN	NaN	NaN	NaN	NaN	N
13	NaN	# Label: E>0.8 = Electrons at >0.8 Mev	NaN	NaN	NaN	NaN	NaN	NaN	NaN	N
14	NaN	# Label: E>2.0 = Electrons at >2.0 Mev	NaN	NaN	NaN	NaN	NaN	NaN	NaN	N
15	NaN	# Label: E>4.0 = Electrons at >4.0 Mev	NaN	NaN	NaN	NaN	NaN	NaN	NaN	N
16	NaN	# Units: Particles = Protons/cm2-s-sr	NaN	NaN	NaN	NaN	NaN	NaN	NaN	N
17	NaN	# Units: Electrons = Electrons/cm2-s-sr	NaN	NaN	NaN	NaN	NaN	NaN	NaN	N
18	NaN	# Source: GOES-15	NaN	NaN	NaN	NaN	NaN	NaN	NaN	N
19	NaN	# Location: W135	NaN	NaN	NaN	NaN	NaN	NaN	NaN	N
20	NaN	# Missing data: -1.00e+05	NaN	NaN	NaN	NaN	NaN	NaN	NaN	N
21	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	N
22	NaN	5-minute GOES-15 Solar Particle and Electron ...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	N
23	NaN	NaN	NaN	NaN	NaN	Modified	Seconds	NaN	NaN	N
24	NaN	# UT	C D	ate	Time	Julian	of the	NaN	NaN	N
25	NaN	# YR	MO	DA	HHMM	Day	Day	P ₁	P	N
26	NaN	2017	8	28	0	57993	0	12	0	N
27	NaN	2017	8	28	5	57993	300	19.4	0	N
28	NaN	2017	8	28	10	57993	600	8.48	0.1	N
29	NaN	2017	8	28	15	57993	900	15.4	0.5	N

Завдання 1.3

Для кожного стовпця порахуйте, скільки в ньому заповнених клітинок (

`RAW.notna().sum()`), і виведіть результат.

Очікуваний результат: 32 числа. Серед них будуть нулі та згустки великих значень. За цими згустками видно, що в аркуші **неодна таблиця, а кілька**, вставлені поруч.

Питання: скільки таблиць в аркуші і які стовпці займає кожна?

```
# ВАШ КОД
```

```
print(RAW.notna().sum())
```

```
0      0
1     7223
2     7202
3     7202
4     7202
5     7203
6     7203
7     7198
8     7198
9     7198
10    7198
11    7198
12    7198
13    7198
14    7198
15      0
16     32
17     26
18     26
19     26
20      0
21      0
22      0
23      0
24     601
25     601
26     608
27     602
28     602
29     602
30     602
31     602
dtype: int64
```

Відповідь 1.3: (скільки таблиць і які стовпці кожна займає)

У аркуші 3 таблиці: Перша займає стовпці 1-14, друга 16-19, третя 24-31

У аркуші 3 таблиці: Перша займає стовпці 1-14, друга 16-19, третя 24-31

Завдання 1.4

Над кожною таблицею є кілька текстових рядків — це службовий опис від тих, хто дані збирав. Виведіть їх і виписіть для кожної таблиці **джерело даних та одиниці вимірювання**.

Це найкорисніші п'ять хвилин роботи: жодної з цих відомостей у самій таблиці немає, і без них ви не знатимете, у чому вимірюєте.

```
# ВАШ КОД print("Таблиця A")
print(RAW.iloc[0:23, 1].dropna())

print("\nТаблиця B")
print(RAW.iloc[13:25, 16].dropna())

print("\nТаблиця C")
print(RAW.iloc[0:8, 26].dropna())
```

Таблиця A

```
0      ftp://ftp.swpc.noaa.gov/pub/lists/particle/
2      :Data_list: 20170901_Gs_part_5m.txt
3      :Created: 2017 Sep 02 0011 UTC
4      # Prepared by the U.S. Dept. of Commerce, NOAA...
5      # Please send comments and suggestions to SWPC...
6      #
7      # Label: P > 1 = Particles at >1 Mev
8      # Label: P > 5 = Particles at >5 Mev
9      # Label: P >10 = Particles at >10 Mev
10     # Label: P >30 = Particles at >30 Mev
11     # Label: P >50 = Particles at >50 Mev
12     # Label: P>100 = Particles at >100 Mev
13     # Label: E>0.8 = Electrons at >0.8 Mev
14     # Label: E>2.0 = Electrons at >2.0 Mev
15     # Label: E>4.0 = Electrons at >4.0 Mev
16     # Units: Particles = Protons/cm2-s-sr
17     # Units: Electrons = Electrons/cm2-s-sr
18     # Source: GOES-15
19     # Location: W135
20     # Missing data: -1.00e+05 22      5-minute      GOES-15 Solar Particle and
Electron ... Name: 1, dtype: object
```

Таблиця B

```
13     ftp://ftp.swpc.noaa.gov/pub/indices/old\_indices...
15     Product: Daily Solar Data      quar_DSD.txt
16     Issued: 0825 UT 04 Oct 2017
17#18     # Prepared by the U.S. Dept. of Commerce, NOAA... 18     #
Prepared by the U.S. Dept. of Commerce, NOAA...
19     # Please send comments and suggestions to SWP...
23     Quarterly Daily Solar Data
Name: 16, dtype: object
```

Таблиця C

0

<http://umtof.umd.edu/pm/crn/>

2

SOHO CELIAS Proton Monitor

3 Solar Wind Parameters for Carrington Rotation ...

4 Proton speed [km/sec]

5 Proton density [protons per cubic centimeter] 7 Measurement
time (Year, Month, Day of Month, D... Name: 26, dtype: object

Відповідь 1.4:

Таблиця	Джерело	Одиниці вимірювання
A	NOAA Space Weather Prediction Center (GOES-15)	Protons/cm2-s-sr, Electrons/cm2-s-sr
B	NOAA Space Weather Prediction Center (Daily Solar Data)	Одиниця сонячного потоку (sfu)
C	SOHO CELIAS Proton Monitor	Швидкість km/sec, густина protons/cm3

Етап 2. Три таблиці в робочому вигляді

У файлі три таблиці. Ось що в них лежить — решту ви вже знайшли самі:

Таблиця	Стовпці аркуша	Що це	Один рядок
A	B–O	вимірювання із супутника GOES-15: потоки частинок різної енергії	5 хвилин
B	Q–T	добовий показник сонячної активності F10.7	доба
C	Y–AF	швидкість і густина сонячного вітру	година

Завдання 2.1

Виділіть кожну таблицю в окремий DataFrame — назвіть їх A, B і C. Дайте стовпцям осмислені назви й перетворіть значення на числа (pd.to_numeric(..., errors='coerce') стане в пригоді).

```
# ВАШ КОД
A = RAW.iloc[26:7226, 1:15].copy()
B = RAW.iloc[27:52, 16:20].copy()
C = RAW.iloc[27:628, 24:32].copy()

A.columns = ['YR', 'MO', 'DA', 'HHMM', 'Julian_Day', 'Sec_of_Day', 'P>1', 'P
B.columns = ['YR', 'MO', 'DA', 'F10.7']
C.columns = ['idx1', 'idx2', 'YR', 'MO', 'DA', 'HH', 'SPEED', 'DENSITY']

for df in [A, B, C]:
    for col in df.columns:
        for col in df.columns:
            df[col] = pd.to_numeric(df[col], errors='coerce')
```

Завдання 2.2

У кожній таблиці дата й час записані **по-різному** й розкидані по кількох стовпцях. Зберіть для кожної таблиці один стовпець `ts` типу `datetime`.

Після цього обов'язково виведіть для кожної таблиці найранішу й найпізнішу дату.

Очікуваний результат: усі три таблиці мають лежати в межах серпня–вересня 2017 року. Якщо у вас вийшов інший рік — придивіться, як записано рік у тій таблиці, і виправте.

```
# ВАШ КОД
A['ts'] = pd.to_datetime(
    A['YR'].astype(int).astype(str) + '-' +
    A['MO'].astype(int).astype(str).str.zfill(2) + '-' +
    A['DA'].astype(int).astype(str).str.zfill(2) + ' ' +
    A['HHMM'].astype(int).astype(str).str.zfill(4),
    format='%Y-%m-%d %H%M'
)

B['ts'] = pd.to_datetime(
    B['YR'].astype(int).astype(str) + '-' +
    B['MO'].astype(int).astype(str).str.zfill(2) + '-' +
    B['DA'].astype(int).astype(str).str.zfill(2)
)

C['ts'] = pd.to_datetime(
    '20' + C['YR'].astype(int).astype(str) + '-' +
    C['MO'].astype(int).astype(str).str.zfill(2) + '-' +
    C['DA'].astype(int).astype(str).str.zfill(2) + ' ' +
    C['HH'].astype(int).astype(str).str.zfill(2) + ':00'
)

print("A:", A['ts'].min(), "-", A['ts'].max())
print("B:", B['ts'].min(), "-", B['ts'].max())
print("C:", C['ts'].min(), "-", C['ts'].max())
```

```
A: 2017-08-28 00:00:00 - 2017-09-21 23:55:00
B: 2017-08-28 00:00:00 - 2017-09-21 00:00:00
C: 2017-08-28 00:00:00 - 2017-09-22 00:00:00
```

Самоперевірка етапу 2. Клітинка має виконатися без помилок.

```
assert len(A) == 7200, 'у таблиці A має бути 7200 рядків, а є %d' % len(A)
assert len(B) == 25, 'у таблиці B має бути 25 рядків, а є %d' % len(B)
assert len(C) == 601, 'у таблиці C має бути 601 рядок, а є %d' % len(C) for
назва, τ in [('A', A), ('B', B), ('C', C)]:
    рік = pd.to_datetime(τ['ts']).dt.year assert рік.min() == 2017 and
    рік.max() == 2017, 'у таблиці %s рік не 2017' % назва
print('Етап 2 пройдено')Етап 2 пройдено
```

Етап 3. Паспорт даних

Завдання 3.1

Для кожної таблиці виведіть: розмір, типи стовпців, кількість пропусків у кожному стовпці, кількість повних дублікатів рядків і описові статистики (`describe()`).

```
# ВАШ КОД
for table_name, table in[('A', A),('B', B),('C', C)]:
    print(f"Аналіз таблиці {table_name}\n")
    print(f"Розмір таблиці: {table.shape}\n")

    cols_info = pd.DataFrame({
        'Тип стовпця': table.dtypes,
        'Кількість пропусків': table.isna().sum()
    })

    print(f"Типи стовпців та кількість пропусків у них:")
    print(cols_info)

    duplicates_count = table.duplicated().sum()
    print(f"\nКількість повних дублікатів рядків: {duplicates_count}\n")

    print(f"Описова статистика таблиці:")
    print(table.describe())
    print("\n\n")
```

Аналіз таблиці A

Розмір таблиці: (7200, 15)

Типи стовпців та кількість пропусків у них:

	Тип стовпця	Кількість пропусків
YR	int64	0
MO	int64	0
DA	int64	0

HHMM	int64	0
Julian_Day	int64	0
Sec_of_Day	int64	0
P>1	float64	3
P>5	float64	3
P>10	float64	3
P>30	float64	3
P>50	float64	3
P>100	float64	3
E>0.8	float64	3
E>2.0	float64	3
ts	datetime64[ns]	0

Кількість повних дублікатів рядків: 0

Описова статистика таблиці:

	YR	MO	DA	HHMM	Julian_Day	Sec_of_
count	7200.0	7200.000000	7200.000000	7200.000000	7200.000000	7200.000
mean	2017.0	8.840000	13.960000	1177.500000	58005.000000	43050.000
min	2017.0	8.000000	1.000000	0.000000	57993.000000	0.000
25%	2017.0	9.000000	7.000000	588.750000	57999.000000	21525.000
50%	2017.0	9.000000	13.000000	1177.500000	58005.000000	43050.000
75%	2017.0	9.000000	19.000000	1766.250000	58011.000000	64575.000
max	2017.0	9.000000	31.000000	2355.000000	58017.000000	86100.000
std	0.0	0.366632	8.775483	692.481902	7.211603	24943.113

	ts
count	7200
mean	2017-09-09 11:57:30 min
2017-08-28 00:00:00	25%
2017-09-03 05:58:45	
50%	2017-09-09 11:57:30
75%	2017-09-15 17:56:15 max
2017-09-21 23:55:00	std
NaN	

Аналіз таблиці B

Розмір таблиці: (25, 5)

Типи стовпців та кількість пропусків у них:

	Тип стовпця	Кількість пропусків
YR	int64	0
MO	int64	0
DA	int64	0
F10.7	int64	0
ts	datetime64[ns]	0

Завдання 3.2 — дивні значення

Подивіться на рядок `min` у таблиці `describe()` для таблиці `C`.

Швидкість і густина — фізичні величини, які **не можуть бути від'ємними**. Знайдіть від'ємні значення, порахуйте, скільки їх, і поверніться до службового опису з завдання 1.4: там написано, що вони означають.

Замініть ці значення на `NaN` і покажіть, як після цього змінилися середнє й мінімум.

Очікуваний результат: два числа «до» і «після» для кожного зі стовпців.

```
# ВАШ КОД
```

```
speed_mean_before = C['SPEED'].mean()
speed_min_before = C['SPEED'].min()
speed_neg_count = (C['SPEED'] < 0).sum()

density_mean_before = C['DENSITY'].mean()
density_min_before = C['DENSITY'].min()
density_neg_count = (C['DENSITY'] < 0).sum()

C.loc[C['SPEED'] < 0, 'SPEED'] = float('nan')
C.loc[C['DENSITY'] < 0, 'DENSITY'] = float('nan')

speed_mean_after = C['SPEED'].mean()
speed_min_after = C['SPEED'].min()

density_mean_after = C['DENSITY'].mean()
density_min_after = C['DENSITY'].min()

print(f"Кількість від'ємних значень у стовпці 'SPEED': {speed_neg_count}")
print(f"Кількість від'ємних значень у стовпці 'DENSITY': {density_neg_count}")

comparison_table = pd.DataFrame({
    'Метрика': [
        'SPEED (Середнє)',
        'SPEED (Мінімум)',
        'DENSITY (Середнє)',
        'DENSITY (Мінімум)'
    ],
    'До': [
        speed_mean_before,
        speed_min_before,
        density_mean_before,
        density_min_before
    ],
    'Після': [
        speed_mean_after,
        speed_min_after,
        density_mean_after,
        density_min_after
    ]
})
```

```
]
}))

print("\nПорівняння метрик:\n")
print(comparison_table.to_string(index=False))
```

Кількість від'ємних значень у стовпці 'SPEED': 8
 Кількість від'ємних значень у стовпці 'DENSITY': 8

Порівняння метрик:

Метрика	До	Після
SPEED (Середнє)	506.547421	513.394604
SPEED (Мінімум)	-1.000000	277.000000
DENSITY (Середнє)	2.907654	2.960371
DENSITY (Мінімум)	-1.000000	0.100000

Відповідь 3.2: (що це зазначення, скільки їх, як вони спотворювали середнє)

-1 - це значення, які свідчать про відсутність даних. У обох стовпцях їх 8. Через наявність -1, середнє було занижене, а мінімумом стовпців було несправжнє значення

Завдання 3.3 — питання на подумати

У таблиці A є стовпці, для яких `describe()` слухняно рахує середнє й стандартне відхилення, хоча ці числа не мають жодного сенсу.

Знайдіть їх і поясніть своїми словами, чому середнє для них безглузде. Підказка: не кожне число в таблиці є результатом вимірювання.

Це питання не має однієї «правильної» відповіді з підручника — на нього ви відповідаєте з власного здорового глузду. Формальне пояснення буде на лекції 3.

```
# ВАШ КОД cols = ['YR', 'MO', 'DA', 'HHMM', 'Julian_Day',
'Sec_of_Day', 'ts'] print(A[cols].describe())
```

	YR	MO	DA	HHMM	Julian_Day	Sec_of_
count	7200.0	7200.000000	7200.000000	7200.000000	7200.000000	7200.000
mean	2017.0	8.840000	13.960000	1177.500000	58005.000000	43050.000
min	2017.0	8.000000	1.000000	0.000000	57993.000000	0.000
min	2017.0	8.000000	1.000000	0.000000	57993.000000	0.000
25%	2017.0	9.000000	7.000000	588.750000	57999.000000	21525.000

50%	2017.0	9.000000	13.000000	1177.500000	58005.000000	43050.000
75%	2017.0	9.000000	19.000000	1766.250000	58011.000000	64575.000
max	2017.0	9.000000	31.000000	2355.000000	58017.000000	86100.000
std	0.0	0.366632	8.775483	692.481902	7.211603	24943.113

Відповідь 3.3: (які це стовпці та чому середнє для них не має сенсу)

Такими стовпцям є усі стовпці що стосуються часу: YR, MO, DA, HHMM, Julian_Day, Sec_of_Day, ts. Обчислення середнього або стандартного відхилення для них немає сенсу, бо це часові мітки, які відображають хронологію дослідження, вони не є вимірюваннями фізичних явищ, "середній" місяць чи день не дають нам ніякої інформації про досліджуваний процес

Етап 4. Об'єднання трьох таблиць в одну

Таблиці мають різний крок у часі: 5 хвилин, година і доба. Щоб працювати з ними разом, потрібна спільна сітка. Беремо **годину**.

Кожна таблиця потребує своєї дії:

A: 5 хвилин → година. Кілька рядків згортаються в один — це `resample` + агрегація.

C: уже година. Просте з'єднання по спільному стовпцю `ts` — `merge`.

B: доба → година. Навпаки, одне значення розтягується на 24 години доби.

Завдання 4.1

Зведіть таблицю A до погодинної сітки. Для кожного потоку порахуйте **два** числа за годину: середнє і максимум.

```
# ВАШ КОД
flux_cols = ['P>1', 'P>5', 'P>10', 'P>30', 'P>50', 'P>100', 'E>0.8', 'E>2.0' ]

A_hourly = A.resample('1h', on='ts')[flux_cols].agg(['mean', 'max' ])
A_hourly.columns = [f"{col}_{stat}" for col, stat in A_hourly.columns ]

A_hourly = A_hourly.reset_index()

A_hourly.head()
```

tsP>1_meanP>1_maxP>5_meanP>5_maxP>10_meanP>10_max

P>30_

0	2017-08-28 00:00:00	9.778333	19.40	0.324917	0.690	0.192917	0.314	0.09
1	2017-08-28 01:00:00	7.994167	13.60	0.291667	0.512	0.156583	0.259	0.07
2	2017-08-28 02:00:00	6.099167	10.80	0.283000	0.479	0.177667	0.321	0.08
3	2017-08-28	4.752500	7.76	0.236833	0.329	0.166333	0.244	0.08

Подальші дії:

Завдання 4.2

У вашому завданні названо агрегат (середнє , максимум , медіана або мінімум).

Візьміть ваш потік за одну добу, порахуйте для нього по годинах ваш агрегат і ще один на ваш вибір, і побудуйте обидві криві на одному графіку.

Питання: чим вони відрізняються і в якій ситуації різниця між ними стане суттєвою

```
# ВАШ КОД
import matplotlib.pyplot as plt

day_data = A_hourly[A_hourly['ts'].dt.date == pd.to_datetime('2017-09-20').date]

plt.figure(figsize=(12,5))

plt.plot(day_data['ts'], day_data['P>5_max'], label='Максимум', color='crimson')
plt.plot(day_data['ts'], day_data['P>5_mean'], label='Середнє', color='navy')

plt.title("Погодинний потік протонів P>5 за 20.09.2017 (Максимум vs Середнє)")
plt.xlabel("Час (години)", fontsize=11)
plt.ylabel("Потік частинок (Protons/cm2-s-sr)", fontsize=11)
plt.grid(True, linestyle=':', alpha=0.7)
plt.legend(fontsize=11)
plt.xticks(rotation=45)

plt.tight_layout()
plt.show()
```



Відповідь 4.2: (чим відрізняються криві та коли це важливо)

Крива максимуму завжди лежить вище за криву середнього, оскільки вона фіксує найсильніші піки потоку за кожну годину. Крива середнього згладжує ці коливання і показує загальний фоновий рівень радіації. Різниця між ними стає суттєвою під час раптових, спалахових подій. Наприклад, якщо протягом години стався сильний, але короткий сплеск радіації, то середнє значення за годину залишиться низьким і приховає небезпеку, тому для оцінки ризиків та радіаційної безпеки важливо знати саме максимальне значення, а не усереднене.

Завдання 4.3

Об'єднайте три таблиці в одну — назвіть її `M`. Виведіть її розмір і всі рядки, у яких є хоча б один пропуск.

Зверніть увагу: таблиці закінчуються не в один і той самий момент. Тому результат залежить від того, який тип об'єднання ви оберете (`how='inner'` чи `how='outer'`). Спробуйте обидва, подивіться, чим відрізняється кількість рядків, і поясніть свій вибір.

Відповідь 4.3: (який тип об'єднання обрано, скільки рядків дає кожен і чому) `inner`(600 рядків) залишає лише ті години, які присутні в усіх трьох таблицях одночасно, `outer`(601 рядок) зберігає абсолютно всі часові мітки з усіх таблиць. Тому в `outer` вийшло на один рядок більше, бо одна з таблиць мала одну часову мітку яку не мають інші. Серед цих двох типів об'єднання було обрано `inner`, щоб мати повні синхронізовані дані з усіх таблиць

Завдання 4.4 — поділ на групи

Щоб далі було що з чим порівнювати, поділіть години на три групи за значенням **максимуму потоку P>10 за годину**:

Група Умова		тиха	менше 10
активна	від 10 до 1000		
дуже активна	1000 і більше		

Додайте до `M` стовпець `група` і виведіть, скільки годин потрапило в кожну.

Це не вигадані числа: саме такі пороги використовує метеослужба космічної погоди NOAA. Чому вони саме такі — питання не сьогоднішньої роботи.

```
# ВАШ КОД
import numpy as np

conditions = [
    M['P>10_max'] < 10,
    (M['P>10_max'] >= 10) & (M['P>10_max'] < 1000),
    M['P>10_max'] >= 1000
]

labels = ['тиха', 'активна', 'дуже активна']

M['група'] = np.select(conditions, labels, default='невідомо')

result = M['група'].value_counts().reset_index()
result.columns = ['Група', 'Кількість годин']

print(result)
```

	Група	Кількість годин
0	тиха	411
1	активна	171
2	дуже активна	18
2	дуже активна	18

Самоперевірка етапу 4.

```
assert M is not None, 'таблиця M не побудована'
assert 'ts' in M.columns, 'у таблиці M потрібен стовпець ts'
assert 'група' in M.columns, 'у таблиці M потрібен стовпець «група»'
assert 590 <= len(M) <= 605, 'очікується близько 600 рядків, а є %d' % len(M)
print('Етап 4 пройдено:', len(M), 'рядків')
```

Етап 4 пройдено: 600 рядків

Етап 5. П'ять графіків — п'ять питань

Графік будується як **відповідь на питання**, а не «щоб був».

Вимоги до кожного графіка:

- підписані обидві осі, з одиницями вимірювання;
- заголовок;
- під графіком, у текстовій клітинці, — **одне речення про дані**. Не «на графіку зображено потік протонів», а твердження на кшталт «більшість годин потік не перевищує 60, але кілька годин дають значення у сотні разів більші».

Не кожне питання має відповідь «так, зв'язок є». Якщо на графіку нічого не видно — так і напишіть. Відсутність зв'язку — це теж результат, і чесно про неї написати важливіше, ніж підбирати ракурс, у якому «щось наче проглядається».

Графік 1. Як розподілені значення вашого потоку?

Побудуйте гістограму вашого потоку (середнє за годину). Порахуйте окремо середнє й медіану цього стовпця.

Якщо гістограма вийде нечитабельною — стовпчик біля нуля і порожньо далі — не залишайте так. Спробуйте побудувати гістограму від логарифма значень (пр. $\log_{10}$) і подивіться, що зміниться.

Питання: чому середнє й медіана так сильно відрізняються?

```

# ВАШ КОД
import matplotlib.pyplot as plt
import matplotlib.pyplot plt

import numpy as np

mean_val = M['P>5_mean'].mean() median_val
= M['P>5_mean'].median()

print(f"Середнє значення P>5: {mean_val:.4f}")
print(f"Медіана P>5: {median_val:.4f}") fig,
axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].hist(M['P>5_mean'], bins=30, color='skyblue', edgecolor='black')
axes[0].axvline(mean_val, color='red', linestyle='--', linewidth=2, label=f'
axes[0].axvline(median_val, color='green', linestyle='--', linewidth=2, label
axes[0].set_title("Звичайна гістограма потоку P>5", fontsize=12)
axes[0].set_xlabel("Потік частинок (Protons/cm2-s-sr)", fontsize=10)
axes[0].set_ylabel("Кількість годин (частота)", fontsize=10)
axes[0].grid(True, linestyle=':', alpha=0.6) axes[0].legend() log_data =
np.log10(M['P>5_mean'])

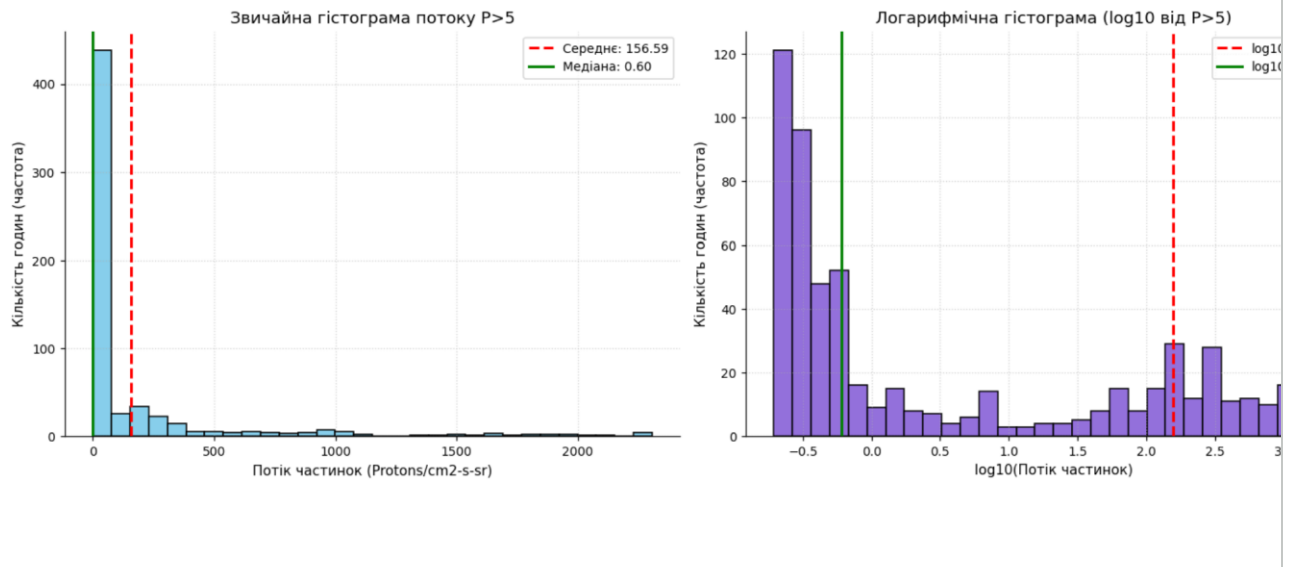
axes[1].hist(log_data, bins=30, color='mediumpurple', edgecolor='black')
axes[1].axvline(np.log10(mean_val), color='red', linestyle='--', linewidth=2
axes[1].axvline(np.log10(median_val), color='green', linestyle='--', linewidth
axes[1].set_title("Логарифмічна гістограма (log10 від P>5)", fontsize=12)
axes[1].set_xlabel("log10(Потік частинок)", fontsize=10)
axes[1].set_ylabel("Кількість годин (частота)", fontsize=10)
axes[1].grid(True, linestyle=':', alpha=0.6) axes[1].legend()

plt.tight_layout() plt.show()

```

Середнє значення $P > 5$: 156.5898

Медіана $P > 5$: 0.6014



Висновок:

На графіку бачимо, що більшість годин потік протонів не перевищує 1 частинки, проте під час поодиноких сонячних спалахів він зростає у тисячі разів (понад 2000). Середнє значення (156.69) і медіана (0.60) сильно відрізняються через асиметрію розподілу та наявність аномальних викидів. Медіана є числом посередині впорядкованого набору даних, тому вона показує типовий рівень потоку, на який припадає більшість годин. А середнє арифметичне збільшується через наявність поодиноких великих спалахів (як от 2000).

Графік 2. Чи відрізняється швидкість вітру між групами годин?

Побудуйте діаграму розмаху (`boxplot`) швидкості сонячного вітру окремо для кожної групи з завдання 4.4.

```

# ВАШ КОД
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(9,6))

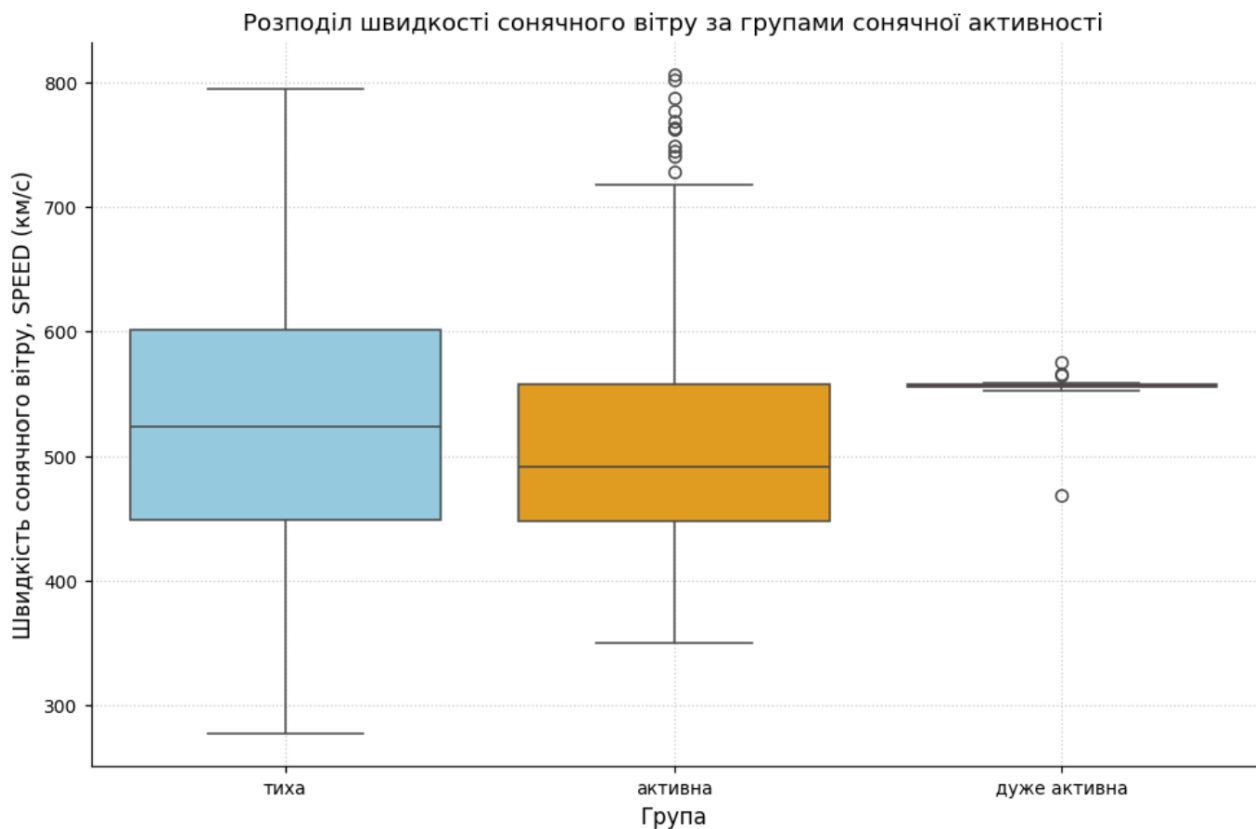
group_order = ['тиха', 'активна', 'дуже активна']

sns.boxplot(
    data=M,
    x='група',
    y='SPEED',
    order=group_order,
    hue='група',
    legend=False,
    palette=['skyblue', 'orange', 'crimson']
)

plt.title("Розподіл швидкості сонячного вітру за групами сонячної активності",
plt.xlabel("Група", fontsize=11)
plt.ylabel("Швидкість сонячного вітру, SPEED (км/с)", fontsize=11)
plt.grid(True, linestyle=':', alpha=0.6)

plt.tight_layout()
plt.show()

```



Висновок:

Чіткої прямої залежності між зростанням сонячної активності та швидкістю вітру немає - медіанна швидкість вітру в усіх трьох групах залишається приблизно однаковою (близько 480–580 км/с). Проте групи сильно відрізняються за розподілом - в «тихій» та «активній» групах спостерігається значно більший розкид значень, ніж у групі «дуже активна», бо вона охоплює лише 18 годин. Також можемо бачити що саме група "активна" має найбільше аномальних сплесків швидкостей (до 800 км/с)

Графік 3. Чи пов'язані змінні вашої пари?

Побудуйте діаграму розсіювання для пари з вашого завдання. Порахуйте коефіцієнт кореляції між ними.

```
# ВАШ КОД
import matplotlib.pyplot as plt
import seaborn as sns

clean_data = M[['F10.7', 'SPEED']].dropna ()
corr_val = clean_data['F10.7'].corr(clean_data['SPEED'])

print(f"Коефіцієнт кореляції Пірсона між F10.7 та SPEED: {corr_val:.4f}")

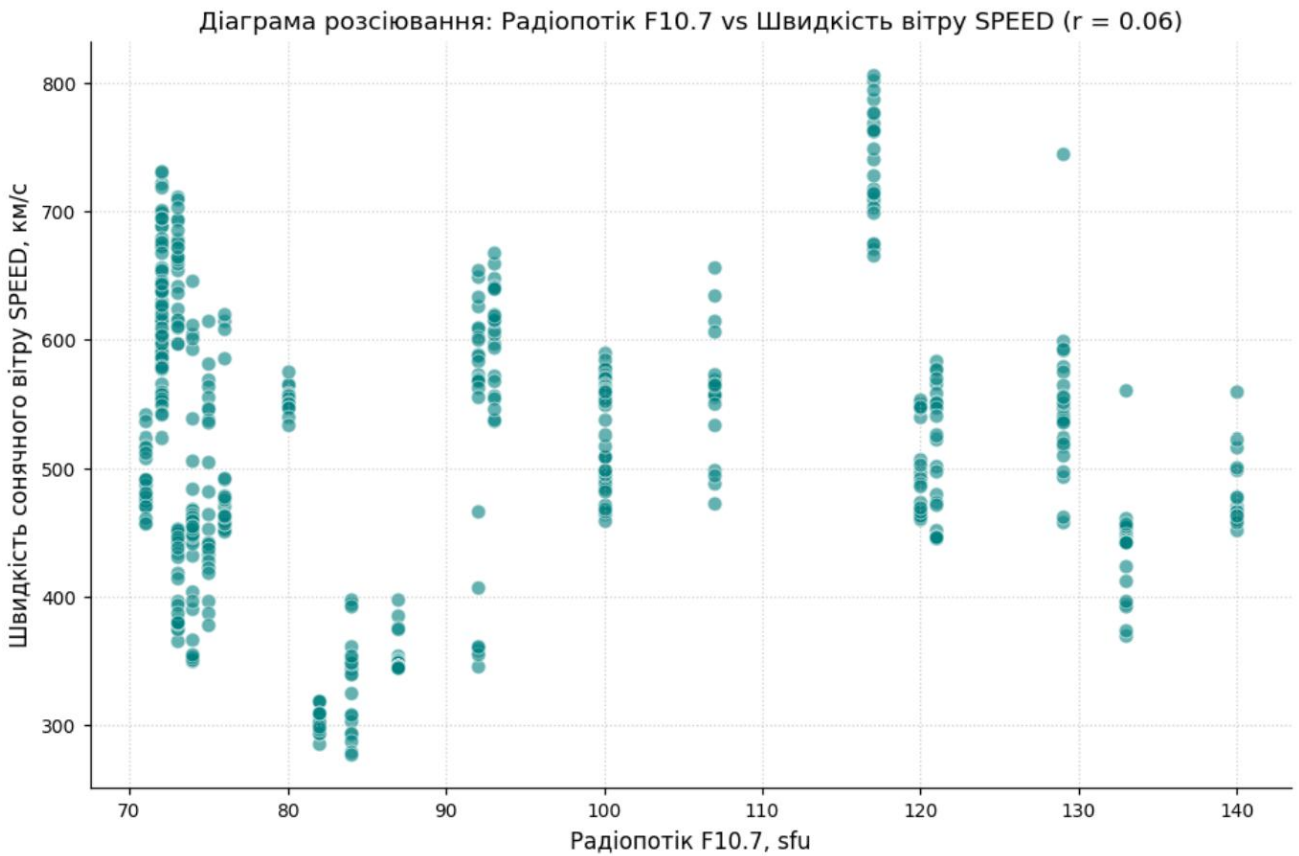
plt.figure(figsize=(9,6))

sns.scatterplot(
    data=clean_data,
    x='F10.7',
    y='SPEED',
    alpha=0.6,
    color='teal',
    edgecolor='w',
    s=50
)

plt.title(f"Діаграма розсіювання: Радіопотік F10.7 vs Швидкість вітру SPEED (r
plt.xlabel("Радіопотік F10.7, sfu", fontsize=11)
plt.ylabel("Швидкість сонячного вітру SPEED, км/с", fontsize=11)
plt.grid(True, linestyle=':', alpha=0.6)

plt.tight_layout()
plt.show()
```

Коефіцієнт кореляції Пірсона між F10.7 та SPEED: 0.0593



Висновок:

Точки на діаграмі розсіювання розміщені хаотично без вираженого тренду, а значення коефіцієнта кореляції Пірсона (0.0593) свідчить про повну відсутність лінійного зв'язку між радіопотоком F10.7 та швидкістю сонячного вітру. Тобто, збільшення радіопотоку ніяк не впливає на швидкість сонячного вітру

Графік 4. Скільки годин у кожній групі?

Побудуйте стовпчикову діаграму кількості годин за групами. Підпишіть кожен стовпчик кількістю годин і відсотком від загальної кількості.

```
# ВАШ КОД
import matplotlib.pyplot as plt
import seaborn as sns
```

```

group_counts = M['група'].value_counts().reindex(['тиха', 'активна',
'дуже а total_hours = len(M)

plt.figure(figsize=(8, 6)) colors = ['skyblue', 'orange', 'crimson'] bars
= plt.bar(group_counts.index, group_counts.values, color=colors, edgeco

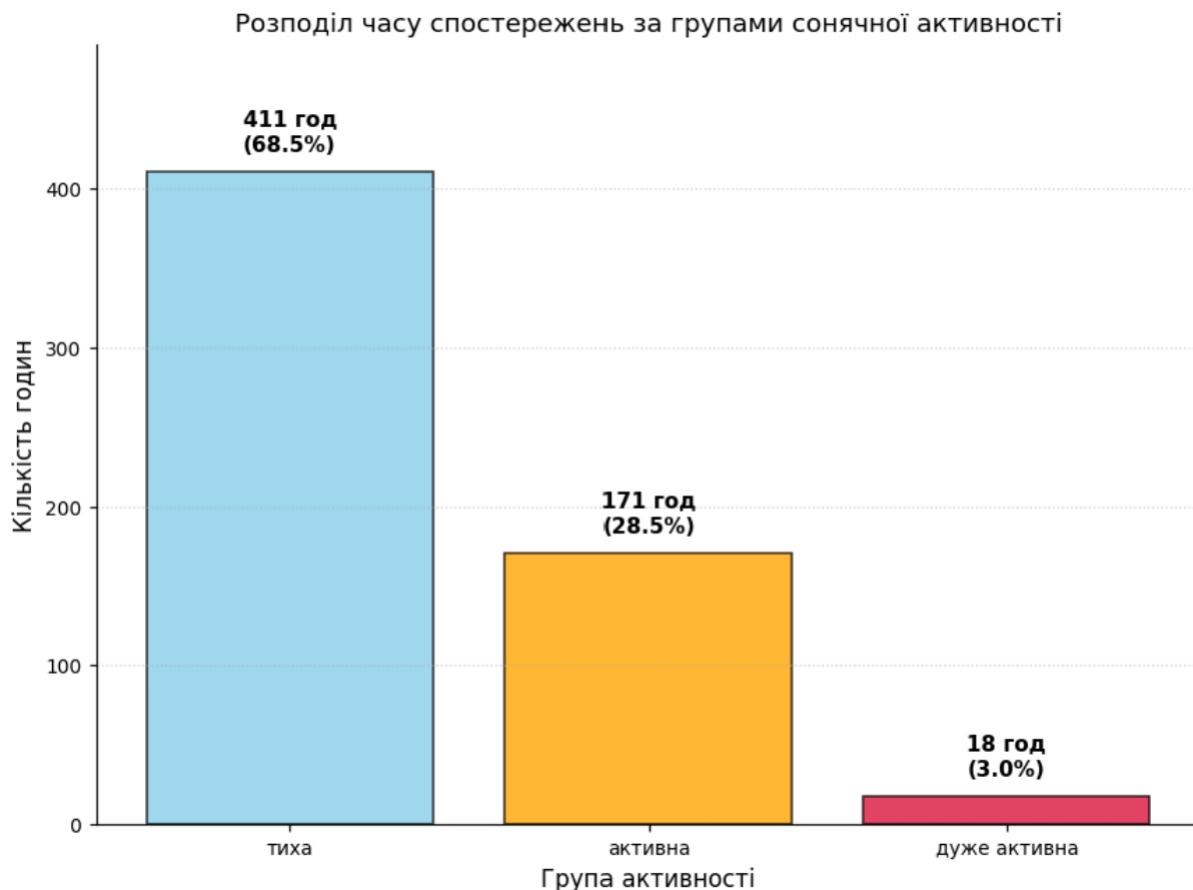
for bar in bars:
    height = bar.get_height()    percentage = (height
/ total_hours) * 100    label_text = f"{int(height)}
год\n({percentage:.1f}%)"

    plt.text(        bar.get_x() +
bar.get_width()/2.,        height +
10,        label_text,
ha='center',        va='bottom',
fontsize=10,        fontweight='bold'
)

plt.title("Розподіл часу спостережень за групами сонячної активності",
fonts plt.xlabel("Група активності", fontsize=11) plt.ylabel("Кількість
годин", fontsize=11) plt.ylim(0, max(group_counts.values) + 80)
plt.grid(axis='y', linestyle=':', alpha=0.6)

plt.tight_layout()
plt.show()

```



Висновок:

Переважну більшість часу спостережень (68.5%, 411 годин) Сонце перебувало у «тихому» стані, на «активну» фазу припало 28.5% часу (171 година), а стан «дуже активна» спостерігався лише протягом 3.0% часу (18 годин).

Графік 5. Як усе це виглядало в часі?

Побудуйте лінійний графік максимуму потоку $P>10$ за годину за весь період спостережень.

Якщо крива виявиться нечитабельною — згадайте, що допомогло в графіку 1.

```
# ВАШ КОД
import matplotlib.pyplot as plt

plt.figure(figsize=(12,5))

plt.plot(M['ts'], M['P>10_max'], color='crimson', linewidth=1.5, label='P>10_m

plt.title("Динаміка максимуму потоку P>10 у часі за весь період спостережень",
plt.xlabel("Дата та час", fontsize=11)
plt.ylabel("Максимум потоку P>10, protons/cm2-s-sr", fontsize=11)
plt.grid(True, linestyle=':', alpha=0.6)
plt.legend()

plt.tight_layout()
plt.show()
```



Висновок: У часовому розподілі максимального потоку $P > 10$ можемо побачити тривалий період спокою до 5 вересня, після чого починається серія сплесків та найсильніший пік у період приблизно з 10 по 12 вересня, після чого потік поступово згасає

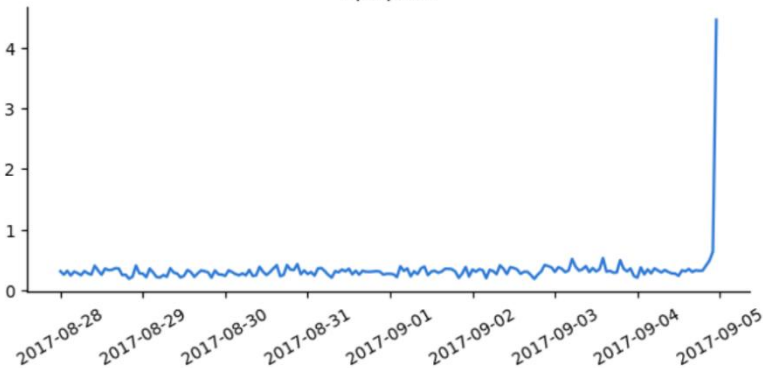
Етап 6. Два графіки одних і тих самих даних

Нижче побудовано два графіки. Дані в них **одні й ті самі** — це той самий стовпець з тієї самої таблиці, жодне число не змінено.

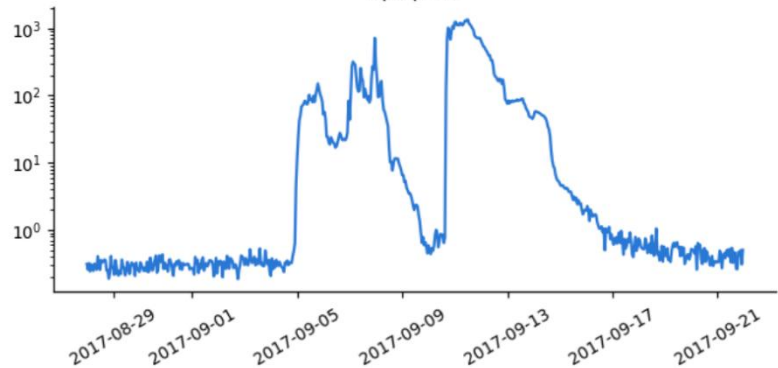
Виконайте клітинку й подивіться на обидва.

```
СТОВПЕЦЬ = 'P>10_max' # <- впишіть назву свого стовпця з МАКСИМУМОМ P>10 за  
вікно = M[(M['ts'] >= '2017-08-28') & (M['ts'] < '2017-09-05')]  
  
fig, ax = plt.subplots(1, 2, figsize=(12, 3.2))  
  
ax[0].plot(вікно['ts'], вікно[СТОВПЕЦЬ], color='#2a78d6')  
ax[0].set_title('Графік 1')  
ax[0].tick_params(axis='x', rotation=30)  
  
ax[1].plot(M['ts'], M[СТОВПЕЦЬ], color='#2a78d6')  
ax[1].set_yscale('log')  
ax[1].set_title('Графік 2')  
ax[1].tick_params(axis='x', rotation=30)  
  
fig.tight_layout()  
plt.show()
```

Графік 1



Графік 2



Завдання 6.1

Дайте відповідь на три питання:

Який висновок про космічну погоду у вересні 2017 року зробить людина, яка бачила **тільки перший** графік?

Який висновок зробить людина, яка бачила **тільки другий**?

Чим ці два графіки відрізняються між собою? Назвіть **дві** відмінності — вони обидві є в кодї вище.

Жодне число в обох графіках не підроблене. Різниця виникла тільки через те, як їх побудували.

Відповідь 6.1:

Висновок з першого графіка: Космічна погода було спокійною протягом вересня, і лише 5 вересня стався один невеликий сплеск (потік зріс близько до 4)

Висновок з другого графіка: У вересні відбулося два сильні сонячні спалахи: перший 5-8 вересня, другий 10-13 вересня, де потік перевищив 1000

Дві відмінності між ними: Перша відмінність полягає у тому, що в першому графіку обрано обмежене вікно часу з 28 серпня по 5 вересня, в той час як у другому графіку показується весь період спостережень. Друга відмінність на першому графіку звичайна лінійна шкала, а на другому - логарифмічна

Завдання 6.2

Уявіть, що вам потрібно **одним графіком** чесно показати, що відбувалося в цьому періоді.

Побудуйте його самі й підпишіть заголовком, який формулює висновок.

Поясніть у клітинці нижче, чому ви обрали саме такий вигляд.

```
# ВАШ КОД
import matplotlib.pyplot as plt

plt.figure(figsize=(11,5))

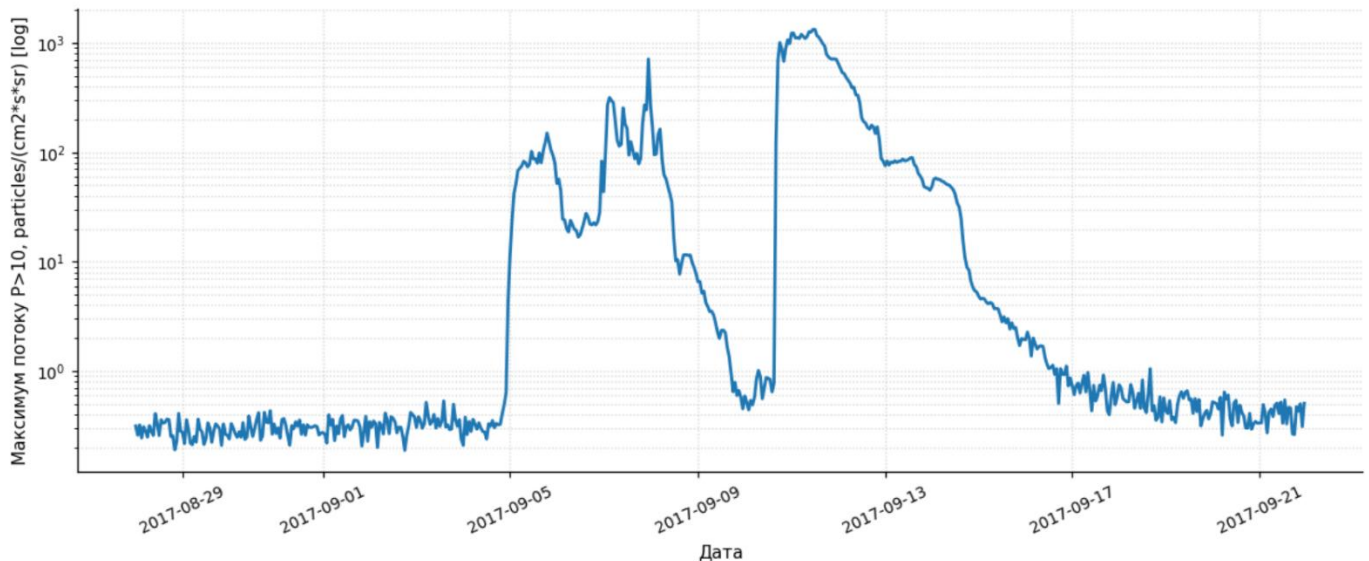
plt.plot(M['ts'], M['P>10_max'], color='#1f77b4', linewidth=1.8, label='P>10')
plt.yscale('log')

plt.title("У вересні 2017 року відбулося два сонячні спалахи: помірний (5-8 вересня) та сильний (10-13 вересня)
          fontsize=11, fontweight='bold', pad=15)

plt.xlabel("Дата", fontsize=10)
plt.ylabel("Максимум потоку P>10, particles/(cm2*s*sr) [log]", fontsize=10)
plt.grid(True, which="both", linestyle=':', alpha=0.5)
plt.xticks(rotation=25)

plt.tight_layout()
plt.show()
```

У вересні 2017 року відбулося два сонячні спалахи: помірний (5-8 вересня) та сильний (10-13 вересня)



Відповідь 6.2: (чому ви побудували графік саме так)

По-перше, було обрано весь проміжок спостереження, щоб не втрачалась жодна важлива подія (як спалахи)

По-друге, потік частинок має великий розмах значень (від менше ніж 1 до 1000), тому була обрана логарифмічна шкала, оскільки вона дозволяє чіткіше показати як дрібні коливання, так і спалахи

Етап 7. Три спостереження та ваше питання

Завдання 7.1

Запишіть **три спостереження** про ці дані, які ви зробили під час роботи.

Спостереження — це твердження, яке спирається на конкретне число або графік з вашого ноутбука. Не «дані цікаві», а, наприклад, «максимальне значення потоку у 2000 разів більше за медіанне, і всі такі години припадають на одні й ті самі три доби».

Для кожного вкажіть, **де саме** у вашій роботі його видно.

Спостереження 1: Рівень потоку $P > 10$ у спокійні дні становить менше 1, але під час найсильнішого спалаху протягом 10-13 вересня він зростав у понад 1300 час найсильнішого спалаху протягом 10-13 вересня він зростав у понад 1300 разів, досягаючи значення більше ніж 1300 (графік 5, графік 6.2)

Спостереження 2: Стан «тихої» сонячної погоди спостерігається майже у 23 рази частіше (411 годин, або 68.5%), ніж стан «дуже активна» (18 годин, 3.0%), що підтверджує рідкісний характер екстремальних спалахів (графік 4)

Спостереження 3: Через рідкісні, але сильні спалахи розподіл потоку $P > 5$ є асиметричним: його середнє значення (156.69) перевищує медіану (0.60) у понад 260 разів (графік 1)

Завдання 7.2 — ваше індивідуальне питання

Дайте відповідь на додаткове питання зі свого завдання. Відповідь має спиратися на обчислення: наведіть код і результат, а потім сформулюйте висновок словами.

```
# ВАШ КОД
daily_mean = M.groupby(M['ts'].dt.date)['P>5_mean'].mean()

max_date = daily_mean.idxmax()
max_value = daily_mean.max()

min_date = daily_mean.idxmin()
min_value = daily_mean.min()

print(f"Максимальне середньодобове значення: {max_value:.3f} (дата: {max_date})")
print(f"Мінімальне середньодобове значення: {min_value:.3f} (дата: {min_date})")
```

Максимальне середньодобове значення: 1693.983 (дата: 2017-09-11)
Мінімальне середньодобове значення: 0.241 (дата: 2017-08-29)

Відповідь 7.2: Індивідуальне питання: У яку добу середнє значення вашого потоку було найбільшим? А найменшим? (потік $P > 5$)

За результатами обчислень, найбільше середньодобове значення потоку $P > 5$ становило 1693.983 та було 11.09.2017 (що відповідає піковій фазі сонячного спалаху), а мінімальне значення значення 0.241 було 29.08.2017, що відповідає фазі спокою

Етап 8. Декларація про використання інструментів штучного інтелекту

Користуватися мовними моделями під час виконання цієї роботи **дозволено**. Це нормальна інженерна практика, і курс про штучний інтелект не має підстав її забороняти.

Обов'язковою є прозорість: ви вказуєте, де саме і як скористалися інструментом — так само, як у науковій статті вказують використане програмне забезпечення. **Задекларована допомога порушенням не є.**

Порушенням є інше: здати роботу, яку ви не можете пояснити. На захисті вас попросять показати конкретний рядок вашого коду, пояснити, чому він саме такий, і внести в нього невелику зміну на місці. Якщо код згенеровано, але ви його розумієте — робота зарахована. Якщо не розумієте — не зарахована, незалежно від того, хто його написав. **Завдання 8.1**

Заповніть словник. Для кожного етапу оберіть одне з формулювань:

- 'ні' — інструментами ШІ не користувався;
- 'синтаксис' — питав про назву функції або синтаксис, код писав сам;
- 'код, перевірено' — код згенеровано, я його прочитав, перевірів і розумію;
- 'текст, перероблено' — згенеровано чернетку тексту, яку я переписав по суті;
- 'текст, як є' — згенерований текст залишено майже без змін.

Останній варіант не заборонений, але прямо впливає на оцінку: бали ставляться за висновки, а не за наявність абзацу.

```
ІНСТРУМЕНТИ_ШІ = 'Gemini'      # напр. 'ChatGPT' або 'не використовував'
```

```
ДЕКЛАРАЦІЯ = {  
    'етапи 1-2, читання файлу та розбір таблиць': 'синтаксис',  
    'етап 3, паспорт даних і дивні значення':      'синтаксис',
```

```

        'етап 4, об'єднання таблиць':      'код, перевірено', 'етап 5,
побудова графіків':      'код, перевірено',
        'етап 5, формулювання висновків':      'текст, перероблено',
        'етапи 6-7, порівняння графіків і спостереження': 'ні',
    }

```

Що саме ви перевірили за згенерованим кодом чи текстом – конкретно:

які числа звірили, що переписали, де знайшли помилку.

ЩО_ПЕРЕВІРЕНО = 'ШІ допомагав з синтаксисом та використанням деяких окремих фу

True означає: я можу пояснити кожен рядок свого коду, відтворити будь-який

True означає: я можу пояснити кожен рядок свого коду, відтворити будь-який

результат цієї роботи та внести зміни в код на вимогу під час захисту.

ПІДТВЕРДЖУЮ = True

--- перевірка повноти декларації: клітинку не редагувати -----
 ДОЗВОЛЕНІ = {'ні', 'синтаксис', 'код, перевірено', 'текст, перероблено', 'текст

```

assert ІНСТРУМЕНТИ_ШІ.strip(), 'Вкажіть інструменти або напишіть «не використо
порожні = [k for k, v in ДЕКЛАРАЦІЯ.items() if not str(v).strip()] assert not
порожні, 'Не заповнено пункти декларації: %s' % ' '.join(порожні) чужі = {v
for v in ДЕКЛАРАЦІЯ.values() if v not in ДОЗВОЛЕНІ} assert not чужі,
'Недопустимі значення: %s. Оберіть з переліку вище.' % ' '.join(чужі) assert
len(ЩО_ПЕРЕВІРЕНО.strip()) >= 120, ('Опишіть конкретніше, що саме ви пе
(зараз %d символів, потрібно щонай
% len(ЩО_ПЕРЕВІРЕНО.strip())) assert ПІДТВЕРДЖУЮ is True, 'Підтвердьте, що
можете пояснити власний код на за

```

```

print('ДЕКЛАРАЦІЯ ПРО ВИКОРИСТАННЯ ІНСТРУМЕНТІВ ШІ')
print('=' * 78) print('%-46s %s' % ('автор:', ПІБ))
print('%-46s %s' % ('інструменти:', ІНСТРУМЕНТИ_ШІ))
print('-' * 78) for k, v in ДЕКЛАРАЦІЯ.items():
    print('%-46s %s' % (k + ':', v))
print('-' * 78)
print('перевірено автором:')
print(ЩО_ПЕРЕВІРЕНО)
print('=' * 78)
print('Автор підтверджує, що може пояснити кожен рядок коду під час захисту.')

```

ДЕКЛАРАЦІЯ ПРО ВИКОРИСТАННЯ ІНСТРУМЕНТІВ ШІ

```

=====
автор: Потاپова Світлана інструменти:
Gemini

```

```

етапи 1-2, читання файлу та розбір таблиць: синтаксис етап 3, паспорт
даних і дивні значення: синтаксис етап 4, об'єднання таблиць:
код, перевірено етап 5, побудова графіків: код,
перевірено етап 5, формулювання висновків: текст, перероблено
етапи 6-7, порівняння графіків і спостереження: ні

```

перевірено автором:

ШІ допомагав з синтаксисом та використанням деяких окремих функцій(1-3 етапи)
=====

Автор підтверджує, що може пояснити кожен рядок коду під час захисту.

Крок 9. Збереження результату та фінальна самоперевірка

Об'єднана таблиця знадобиться вам у лабораторних роботах № 3 і № 4 — збережіть її й не втрачайте до кінця семестру.

```
M.to_csv('master_hourly.csv', index=False)
```

```
print('збережено:', M.shape) збережено:
```

```
(600, 21)
```

```
# --- фінальна самоперевірка -----
перевірки = {
    'ПІБ і варіант заповнено':      bool(ПІБ) and bool(ВАРІАНТ),
    'таблиця А (7200 рядків)':      len(A) == 7200,
    'таблиця В (25 рядків)':        len(B) == 25,
    'таблиця С (601 рядок)':        len(C) == 601,
    'таблиці об'єднано':            M is not None and 590 <= len(M) <= 605
    'стовпець «група» є':          'група' in M.columns,
    'усі дати у 2017 році':         int(pd.to_datetime(M['ts']).dt.year.min() == 2017),
    'декларацію заповнено':         bool(ІНСТРУМЕНТИ_ШІ.strip()) and ПІДТВ
} for k, v in перевірки.items(): print(('OK   '
if v else 'НЕ ОК') + '   ' + k)

if all(перевірки.values()):
    print('\nТехнічна частина роботи виконана.') print('Тепер переконайтеся,
що заповнені ВСІ клітинки з відповідями та в print('і виконайте Kernel
→ Restart & Run All перед здачею.')
```

```
OK      ПІБ і варіант заповнено
OK      таблиця А (7200 рядків)
OK      таблиця В (25 рядків)
OK      таблиця С (601 рядок)
OK      таблиці об'єднано
OK      стовпець «група» є
OK      усі дати у 2017 році
OK      декларацію заповнено
```

Технічна частина роботи виконана.

Тепер переконайтеся, що заповнені ВСІ клітинки з відповідями та висновками,
і виконайте Kernel → Restart & Run All перед здачею.